



UNIVERSIDADE ESTADUAL DE CAMPINAS

INSTITUTO DE COMPUTAÇÃO

MO443 - INTRODUÇÃO AO PROCESSAMENTO DE IMAGEM DIGITAL

PROF^º. DR^º. HÉLIO PEDRINI

Trabalho 4

Luís Antônio Almeida Lima Vieira
RA: 221045

23 de junho de 2026

1 Introdução

Neste trabalho foram exploradas transformações geométricas de escala e rotação de uma imagem, assim como o uso das funções de interpolação por vizinho mais próximo, bi-linear, bi-cúbica e por polinômio de Lagrange, de forma a obter os valores dos *pixels* após transformações. Todos os códigos desenvolvidos nesta etapa, foram feitos sem o uso de funções prontas de bibliotecas de processamento de imagens. Em uma segunda parte, foram aplicadas técnicas de detecção de pontos de interesse para registrar um par de imagens e criar uma imagem panorâmica formada pela ligação entre as imagens após sua correspondência.

2 Materiais e Métodos

Para desenvolvimento dos códigos, foi utilizado o sistema operacional *Ubuntu 22.04* na versão *WSL (Windows Subsystem for Linux)*, e nele, foi criado um ambiente específico a partir do *Miniconda* versão 25.9.1 [1]. Foram instalados os seguintes pacotes, mostrados na Tabela 1.

Tabela 1: Pacotes utilizados no Trabalho 1.

Pacote	Versão
python	3.13.9
numpy [2]	2.4.3
matplotlib [3]	3.10.8
opencv [4]	4.12.0

Para a realização dos experimentos da parte 1 (transformações geométricas), foram utilizadas imagens no formato *PNG*, obtidas a partir da página https://www.ic.unicamp.br/~helio/imagens_png/ [5]. Já para a parte 2 (registro), foram utilizadas imagens no formato *JPEG (Joint Photographic Experts Group)* da página https://www.ic.unicamp.br/~helio/imagens_registro/ [6]. Os enunciados da atividade estão disponíveis em <http://www.ic.unicamp.br/~helio/disciplinas/MO443/trabalho3.pdf> [7]. As imagens de entrada foram salvas no diretório *images*. No arquivo *transformações_geométricas.py* foram desenvolvidas as funções de transformação da parte 1, enquanto que os códigos da parte 2 foram desenvolvidos no arquivo *registro.py*.

2.1 Transformações Geométricas

Nesta etapa, foram desenvolvidas funções em *transformações_geométricas.py*, de modo a realizar uma transformação de rotação por um ângulo em graus em ponto flutuante (Ex: 31.5°), ou uma transformação de escala, também por um fator em ponto flutuante. Como especificado, a cada execução, o programa deveria realizar, na imagem de entrada, apenas uma transformação geométrica (escala ou rotação). Não foram utilizados comandos de bibliotecas de processamento de imagens prontos para as operações necessárias. Sendo assim, a implementação seguiu a formulação apresentada a seguir, utilizando comandos vetorizados pelo *numpy* [2].

Para a execução do programa, as variáveis de entrada devem ser passadas pelo terminal, pelo comando a seguir.

```
transformações_geométricas.py [-a ângulo] [-e fator de escala] [-d largura altura]
                             [-m interpolação] [-i imagem de entrada] [-o imagem de saída]
```

onde:

- -a: ângulo de rotação medido em graus no sentido anti-horário (*float*);
- -e: fator de escala (*float*);
- -d: dimensão da imagem de saída em *pixels* (largura altura), com strings de inteiros. Ex.: 225 225;
- -m: nome do método de interpolação utilizado (*string*) ((*near_neighbor*, *bilinear*, *bicubic* or *lagrange*));
- -i: caminho da imagem de entrada no formato PNG (*string*);
- -o: caminho da imagem de saída no formato PNG (após a transformação geométrica) (*string*).

Todos estes comandos são opcionais, com o valor padrão *None* caso nada seja fornecido. Como foi especificado, para realizar apenas uma transformação (escala ou rotação) por execução, se o fator de escala (-e) e o ângulo (-a) forem passados, assim como no casos em que estes dois parâmetros não são fornecidos, o programa retorna a imagem original com um aviso da necessidade de fornecer os parâmetros da maneira esperada.

Com os parâmetros de entrada coletados. Se um caminho de entrada for fornecido, a imagem de entrada é lida pelo *opencv* [4] e a execução segue chamando a função *apply_transform*, que recebe a imagem (*numpy array*) e os parâmetros de fator de escala, dimensão de saída e método de interpolação, retornando a imagem transformada. Se caso o valor da dimensão de saída for passado, a imagem obtida terá a dimensão fixada especificada, com os valores dos *pixels* não existentes após a transformação sendo 0 (preto). Caso contrário, a dimensão é fixada para a mesma dimensão da imagem de entrada. Se caso -d for passado como *max max* a dimensão da escala é dada como os valores inteiros da multiplicação das dimensões de entrada pelo fator no caso de escala. O caso em que a multiplicação se torna 0 ou negativa, é tratado truncando a dimensão mínima de escala para 1.

Como as operações de transformação são aplicadas nas coordenadas dos *pixels* e não nos valores em si, para evitar um *loop*, foi utilizada a função *meshgrid* [8] do *numpy* [2] para gerar a grade de coordenadas dada a dimensão da imagem de saída. Nesta grade, foram aplicadas as transformações rotação e escala de forma indireta (da imagem de saída para a imagem de entrada). A necessidade do mapeamento indireto ao invés do direto (mais intuitivo), se da nos problemas de discretização da imagem, que ocorrem no mapeamento direto e levam diferentes pontos da imagem original serem mapeados para um mesmo ponto da imagem de saída, resultando em falhas na imagem transformada [9].

Como apenas uma transformação é passada por vez, o código detecta se deve ser realizada uma escala ou rotação (mais detalhes serão discutidos nas sessões 2.1.1 e 2.1.2). Após o mapeamento inverso da transformação, serão obtidas as coordenadas da imagem de entrada referente a cada pixel de saída após a transformação. Essas coordenadas obtidas são valores contínuos em ponto flutuante, e para encontrar os *pixels* da imagem original (coordenadas inteiras) que se referem a esta posição, são realizadas operações de interpolação, discutidas a partir de 2.1.3). Essas operações irão retornar os valores dos *pixels* da imagem de saída após a transformação, encontrando seu valor na posição da imagem original antes da transformação.

Como as operações de interpolação levam em conta valores de *pixels* vizinhos, os casos de borda precisam ser tratados, para não ocorrer erros em operações em índices inexistentes. Como solução, foi definido que os valores fora da dimensão da imagem sejam truncados pelo valor da borda, permitindo o cálculo das aproximações sem erros de execução. Foi definido que na imagem final, *pixels* que estejam fora da imagem transformada assumam valor zero (preto). Com isso, foi definida uma máscara indo da posição mínima aceitável definida como -1 e a máxima sendo como as dimensões + 1. Com isso, mantém as posições imediatamente seguintes ou anteriores à borda com valor de *pixel* refletido, e os demais fora da imagem como 0. A imagem de saída é retornada com *pixels* convertidos para *uint8* e truncados de 0 a 255. De forma às operações funcionarem tanto para imagens coloridas (3 canais) e monocromáticas (1 canal), durante as interpolações é aplicada a operação *reshape(-1,1)* do *numpy*, apenas para o caso monocromático, para criação da dimensão de canal, com o valor 1.

2.1.1 Transformação de Escala

Na transformação de escala (função *scale*, um fator de escala é aplicado, ampliando ou reduzindo a imagem. Se caso um valor negativo for passado, foi definido para ser considerado o módulo, pois escalas negativas não fazem sentido nessa implementação. A função desenvolvida, portanto, aplica a escala de forma indireta como discutido anteriormente, recebendo as coordenadas da imagem de saída e mapeando para a imagem de entrada pela simples divisão pelo fator de escala. Com isso, são recebidos os *arrays* das coordenadas da imagem de saída e o fator de escala, retornando as coordenadas da imagem de entrada referentes a cada índice da imagem de saída após a transformação. Portanto:

$$x_i = x_o / \text{fator} \quad (1)$$

$$y_i = y_o / \text{fator} \quad (2)$$

2.1.2 Transformação de Rotação

Na transformação de rotação (função *rotate*), um ângulo em graus é passado, e uma rotação com relação ao centro da imagem é aplicada no sentido anti-horário. Com isso, assim como no caso da escala, as coordenadas de saída são fornecidas, assim como o ângulo de rotação em ponto flutuante, que pode ser negativo e maior que 360° sem prejuízo para o funcionamento, e as dimensões das imagens de entrada e saída, retornando as coordenadas da imagem de entrada referentes às coordenadas de saída após a transformação.

Nesta função, o ângulo recebido é convertido para radianos e as coordenadas centrais de entrada e saída são calculadas através das dimensões das imagens, pela divisão de x e y por 2. Considerando que a matriz de rotação (Equação 3), rotaciona com relação à origem, que em imagens está no canto superior esquerdo, para realizar a rotação em relação ao centro, é necessário aplicar uma transformação de translação (Equação 4) utilizando os valores das coordenadas do centro, de forma que o ponto (0,0) se localize no centro da imagem de saída. Portanto, é aplicada a combinação de translações e rotação da Equação 5, obtendo a transformação da Equação 6.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (3)$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (4)$$

$$M = T(c_x^{in}, c_y^{in}) \cdot R(\theta) \cdot T(-c_x^{out}, -c_y^{out}) \quad (5)$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (6)$$

Em que $T(-c_x^{out}, -c_y^{out})$ é a aplicação da Equação 4 para transladar as coordenadas no valor das coordenadas do centro da imagem de saída, deslocando a posição (0,0) para o centro, movendo o centro para a origem. $R(\theta)$ é a matriz de rotação da Equação 3, que rotacionará em θ com relação à origem. Já $T(c_x^{in}, c_y^{in})$ é o termo que move a

origem da imagem de entrada para o centro, deslocando positivamente pelas coordenadas do centro da imagem original, necessário para a correspondência com as coordenadas originais. Vale ressaltar que essas operações matriciais ocorrem em coordenadas homogêneas, então foi necessário adicionar às coordenadas mais uma dimensão de 1s. Após a operação é retornado o sistema de coordenadas original x e y , obtendo assim o valor das coordenadas de entrada referentes à saída transformada.

2.1.3 Interpolação por Vizinhos próximos

Esta interpolação, assim como as outras, tem como objetivo ler uma coordenada contínua, proveniente de uma transformação, e obter o valor do *pixel* da coordenada relativa à imagem original. Nesta técnica, o valor de intensidade de um *pixel* (x', y') na imagem reamostrada terá o mesmo valor do *pixel* que estiver mais próximo da posição ocupada pelo *pixel* (x, y) na imagem original, conforme a Figura 1.

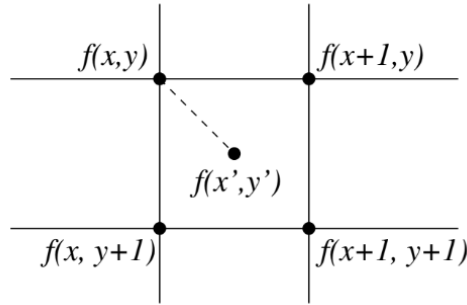


Figura 1: Interpolação pelo vizinho mais próximo.

A interpolação pode ser expressa por:

$$f(x', y') = \begin{cases} f(x, y), & \text{para } d_x < 0.5 \text{ e } d_y < 0.5, \\ f(x + 1, y), & \text{para } d_x \geq 0.5 \text{ e } d_y < 0.5, \\ f(x, y + 1), & \text{para } d_x < 0.5 \text{ e } d_y \geq 0.5, \\ f(x + 1, y + 1), & \text{para } d_x \geq 0.5 \text{ e } d_y \geq 0.5. \end{cases} \quad (7)$$

Em que d_x e d_y correspondem às distâncias nas direções x e y , respectivamente entre os pontos (x', y') e (x, y) , ou seja, apenas as partes fracionárias dos pontos (x', y') .

$$d_x = x' - x = x' - \lfloor x' \rfloor \quad (8)$$

$$d_y = y' - y = y' - \lfloor y' \rfloor \quad (9)$$

A equação 7 pode ser reescrita como:

$$f(x', y') = f(\text{round}(x'), \text{round}(y')) \quad (10)$$

em que `round` é uma função que aproxima um número para seu valor inteiro mais próximo.

Esta interpolação é realizada pela função *near_neighbor* desenvolvida, que recebe a imagem original e as coordenadas x e y da imagem original após a transformação. Os valores recebidos são arredondados pela função *round*, para a obtenção das posições vizinhas, com as distâncias em x e y calculadas pela diferença entre os valores recebidos e os arredondados. Para possibilitar o cálculo pelo *numpy*, foi feita a operação *reshape*, para obtenção de uma dimensão extra, funcionando para imagens multicanais. A coordenada do vizinho mais próximo foi obtida pela função *where* do *numpy* [2] em que distâncias em x (dx) maiores que 0.5 são mapeadas para a coordenada $x + 1$ ou mantidas em x caso contrário. A mesma operação ocorre para y . Com as coordenadas obtidas, o valor do *pixel* é o valor na posição destas coordenadas na imagem original.

2.1.4 Interpolação Bilinear

A interpolação bilinear utiliza uma média ponderada de distância dos quatro *pixel* vizinhos mais próximos para determinar a intensidade de cada *pixel* (x', y') na imagem transformada, como mostrado na figura 2.

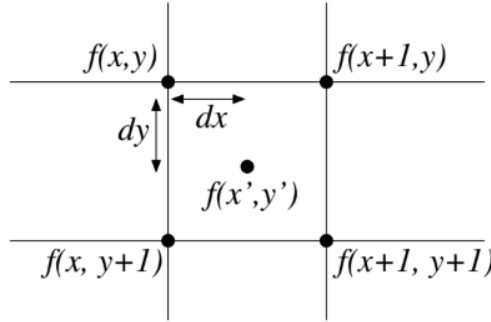


Figura 2: Interpolação Bilinear.

A interpolação é dada pela Equação 11.

$$\begin{aligned}
 f(x', y') = & (1 - d_x)(1 - d_y) f(x, y) \\
 & + d_x(1 - d_y) f(x + 1, y) \\
 & + (1 - d_x)d_y f(x, y + 1) \\
 & + d_x d_y f(x + 1, y + 1).
 \end{aligned} \tag{11}$$

Esta interpolação é realizada pela função *bilinear* desenvolvida, que recebe a imagem original e as coordenadas x e y da imagem original após a transformação. Assim como a interpolação de vizinhos próximos ocorre o arredondamento e o cálculo das distâncias para os vizinhos dx e dy . Com isso, foi implementada a Equação 11, ponderando os valores dos *pixels* com os valores dos vizinhos, obtendo o valor do *pixel* nas coordenadas referentes à imagem original.

2.1.5 Interpolação Bicúbica

A interpolação bicúbica (função *bicubic*) utiliza uma vizinhança de 4×4 pontos ao redor do ponto em questão para calcular seu valor de intensidade, como mostrado na figura 3. Uma função comum para calcular as intensidades do *pixel* na imagem interpolada é a função *B-spline* cúbica, definida na Equação 12.

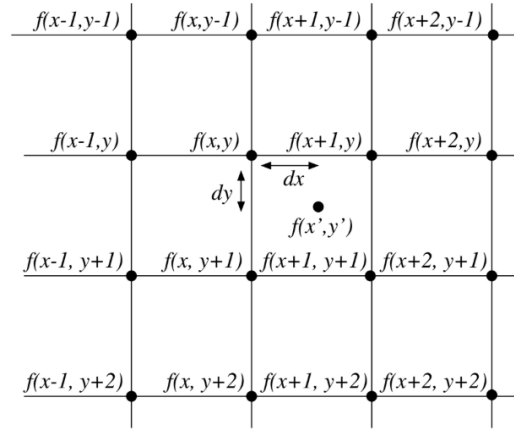


Figura 3: Interpolação Bicúbica.

$$f(x', y') = \sum_{m=-1}^2 \sum_{n=-1}^2 f(x+m, y+n) R(m-d_x) R(d_y-n) \quad (12)$$

$$R(s) = \frac{1}{6} [P(s+2)^3 - 4P(s+1)^3 + 6P(s)^3 - 4P(s-1)^3] \quad (13)$$

$$P(t) = \begin{cases} t, & t > 0 \\ 0, & t \leq 0 \end{cases} \quad (14)$$

Com isso, foi realizada uma implementação semelhante às anteriores, com alteração na fórmula para obter os valores dos *pixels* agora pela Equação 12. Funções auxiliares foram criadas para as Equações 13 e 14.

2.1.6 Interpolação por Polinômios de Lagrange

O último método, utiliza uma vizinhança de 4×4 pontos para calcular o valor de intensidade de um *pixel* (x', y') por polinômios de Lagrange, definida na Equação 15. Este método foi implementado pela função *lagrange*, com a mesma raiz das outras funções de interpolação, alterando apenas a função de cálculo polinomial.

$$f(x', y') = -\frac{d_y(d_y-1)(d_y-2)}{6} L(1) + \frac{(d_y+1)(d_y-1)(d_y-2)}{2} L(2) \\ - \frac{d_y(d_y+1)(d_y-2)}{2} L(3) + \frac{d_y(d_y+1)(d_y-1)}{6} L(4) \quad (15)$$

$$L(n) = -\frac{d_x(d_x-1)(d_x-2)}{6} f(x-1, y+n-2) \\ + \frac{(d_x+1)(d_x-1)(d_x-2)}{2} f(x, y+n-2) \\ - \frac{d_x(d_x+1)(d_x-2)}{2} f(x+1, y+n-2) \\ + \frac{d_x(d_x+1)(d_x-1)}{6} f(x+2, y+n-2) \quad (16)$$

Foi criada uma função auxiliar para o cálculo da Equação 16.

2.2 Registro de Imagens

Nesta etapa, uma imagem panorâmica foi gerada a partir da correspondência de um par de imagens. Pontos de interesse foram detectados e associados para o registro das imagens, de acordo com as técnicas mostradas a seguir.

Toda a implementação desta técnica de registro foi realizada no módulo *registro.py*. Para a execução do programa, as variáveis de entrada devem ser passadas pelo terminal, pelo comando a seguir.

```
registro.py -i1 caminho da imagem 1 -i2 caminho da imagem 2 [-l limiar de correspondência]
[-o diretório para salvar a imagem panorâmica] [-d nome do descritor de pontos de interesse]
```

onde:

- -i1: Caminho da imagem 1 (*string* obrigatória);
- -i2: Caminho da imagem 2 (*string* obrigatória);
- -l: limiar de correspondência (float, definido por padrão como 0.5, baseado em testes empíricos). Este valor indica o quão permissivo o algoritmo é para determinar uma correspondência entre regiões. São considerados como correspondentes, valores de correlação acima deste limiar;
- -o: diretório para salvar as imagens panorâmicas (string). Por padrão é salvo no mesmo diretório do código;
- -d: Nome da técnica de obtenção de descritores de pontos de interesse (*string*), entre *SIFT* (Scale Invariant Feature Transform) (definido como padrão) [10], *BRIEF* (Robust Independent Elementary Features) [11] e *ORB* (Oriented FAST, Rotated *BRIEF*) [12].

Foi desenvolvida uma função (*registry*), que recebe os parâmetros definidos na execução e executa os seguintes passos para a geração da imagem panorâmica e dos pontos correspondentes:

1. Conversão das imagens coloridas de entrada em imagens em níveis de cinza.
2. Detecção de pontos de interesse e extração de descritores locais invariantes, com o uso de um dos descritores dentre entre *SIFT* (Scale Invariant Feature Transform) [10], *BRIEF* (Robust Independent Elementary Features) [11] e *ORB* (Oriented FAST, Rotated *BRIEF*) [12].
3. Cálculo da similaridade entre os descritores das duas imagens por meio da Correlação Cruzada Normalizada (valores de -1 a 1).
4. Seleção das melhores correspondências entre os descritores, a partir do limiar passado como parâmetro, considerando como correspondentes valores acima deste limiar. Foi definido um padrão de 0.5.
5. Estimação da matriz de homografia utilizando o algoritmo *RANSAC* (*RANdom SAmple Consensus*) [13] por meio da função *cv2.findHomography* [14] (o cálculo da matriz de homografia requer o uso de, no mínimo, 4 pontos de correspondência).
6. Realização da transformação de perspectiva utilizando a função *cv2.warpPerspective* [15].
7. Combinação das imagens alinhadas para gerar a panorâmica final.

8. Exibição das correspondências encontradas desenhando retas entre os pontos correspondentes.

O primeiro passo, foi realizado pela função `cv2.cvtColor` do *OpenCV* [4], com o argumento `cv2.COLOR_BGR2GRAY`, para obtenção de versões monocromática das imagens de entrada.

No segundo passo, para a detecção de pontos de interesse e obtenção de descritores locais invariantes, foram testados os seguintes descritores, implementados na função `detect_and_descript`:

- **SIFT (Scale Invariant Feature Transform)** [10]: Este algoritmo detecta e descreve pontos de interesse por meio de gradientes locais da imagem. Produz descritores robustos à escala, rotação e iluminação. Foi implementado por meio do *OpenCV*, por meio da classe `cv2.SIFT_create`, executando o método `detectAndCompute`, passando cada imagem como parâmetro, e mantendo os outros argumentos padrões. Essa função retorna uma lista de pontos de interesse e um *array* com os descritores.
- **BRIEF (Robust Independent Elementary Features)** [11]: Produz descritores binários comparando intensidades entre pares de *pixels* de uma região. Não possui robustez para rotação e escala. Como o *BRIEF* é apenas um descritor, para a detecção de pontos de interesse, foi utilizado uma combinação com o FAST (Features from Accelerated Segment Test) [16] para a detecção de pontos de interesse. A classe `cv2.FastFeatureDetector_create()` com o método `detect`, retorna a lista de pontos de interesse. Os descritores foram obtidos com a classe `cv2.xfeatures2d.BriefDescriptorExtractor_create` pelo método `compute`, que ao entrar a imagem e a lista de pontos de interesse, retorna os descritores.
- **ORB (Oriented FAST, Rotated BRIEF)** [12]: Este método combina o FAST com uma versão orientada do *BRIEF*, introduzindo invariância à rotação, escala com um custo computacional inferior ao *SIFT*. Foi implementado pela classe `cv2.ORB_create`, utilizando o método `detectAndCompute`, assim como no *SIFT*.
- **SURF (Speed Up Robust Feature)** [17]: Foi realizado um teste com esse método, porém na versão utilizada do *OpenCV*, este algoritmo não está disponibilizado por restrições de patente, Este método é uma alternativa otimizada ao *SIFT*;

No terceiro passo, foi definida a Correlação Cruzada Normalizada (Coeficiente de Correlação), para medir a similaridade entre descritores. Neste caso, a máxima similaridade é definida como 1 e a mínima -1. Esta métrica é independente da escala de intensidade da imagem. A formulação do coeficiente de correlação é dado pela Equação 17.

$$CCN(u, v) = \frac{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f(x, y) - \bar{f}] [w(x - u, y - v) - \bar{w}]}{\sqrt{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f(x, y) - \bar{f}]^2 [w(x - u, y - v) - \bar{w}]^2}} \quad (17)$$

A implementação deste cálculo foi realizada pela função `compute_descriptor_similarity`, que recebe dois conjuntos de descritores e retorna a matriz de correlação entre todos os pares de descritores e uma lista de objetos `cv2.DMatch` do *OpenCV*, que representam o valor de similaridade de cada *match*. Todo o cálculo de similaridade foi realizado de maneira vetorizada, removendo as médias dos descritores, calculando a norma pelo *numpy* e dividindo os descritores centralizados pela norma. O cálculo dos coeficientes de correlação entre todas as correspondências entre os descritores normalizados, foi feito pela equação matricial de produto, obtendo, assim a matriz de correlação. Por fim, para cada descritor da primeira imagem, é selecionado o descritor da segunda imagem que apresenta o maior coeficiente de correlação. Essas correspondências são armazenadas em objetos `cv2.DMatch`, contendo os índices dos descritores correspondentes.

No quarto passo, a função `get_best_pairs`, recebe a lista de correspondências gerados no passo anterior, a matriz de correlação e o limiar definido pelo usuário na execução do programa principal. A lista de correspondência é percorrida, e apenas os índices que possuem coeficiente de correlação maiores que o limiar são considerados para a sequência como correspondências.

No quinto passo, é realizada a estimação da matriz de homografia pelo algoritmo *RANSAC* (*RANdom SAmple Consensus*) [13] por meio da função `cv2.findHomography` [14] do *OpenCV*. Foi desenvolvida a função `estimate_homography`, que recebe os pontos de interesse de ambas as imagens e a lista dos índices das correspondências válidas, gerada pela etapa anterior. Conforme o enunciado, a matriz de homografia necessita de no mínimo 4 pontos de correspondência. Casos que não cumprem esse requisito causam um erro na execução do programa. Com isso, os pontos de interesse pertencentes aos melhores índices, são passados para `cv2.findHomography` com o parâmetro de método `cv2.RANSAC`. Esta função retorna a matriz de homografia, mostrada na tela e a máscara binária informando quais correspondências foram utilizadas para a matriz.

No sexto passo, é aplicada a função `cv2.warpPerspective` [15], para realizar a transformação em perspectiva utilizando a matriz de homografia, mapeando cada *pixel* da imagem 1 para uma nova posição, ocasionando uma distorção em perspectiva da imagem. Como a matriz de homografia é formada pelas transformações necessárias para juntar os pontos correspondentes, ao aplicar esta na imagem 1, as correspondências ficarão na mesma posição de seu par, neste caso, a imagem 2. Com isso, esta função retorna a imagem 1 na posição já alinhada para ser unida à imagem 2.

Para aplicar corretamente as transformações, foi criada a função `get_perspective`, que recebe a imagem 1, a imagem 2 e a matriz de homografia. Como a função `cv2.warpPerspective` pede além da imagem a ser transformada e da matriz de homografia, a dimensão de saída, para evitar um valor fixo de saída que possa causar cortes, foi feita uma implementação que estima a dimensão necessária para a panorâmica total. Primeiramente, são salvas as coordenadas dos quatro cantos de ambas as imagens. Uma transformação de perspectiva prévia dos cantos da imagem 1 é feita por meio de `cv2.perspectiveTransform`, e estes cantos transformados são unidos com os cantos da imagem 2, permitindo obter as dimensões máximas e mínimas da imagem panorâmica final. As diferenças entre estes máximos e mínimos formam a dimensão da saída. Além disso, uma transformação de translação por *x* e *y* mínimos, foi aplicada `cv2.warpPerspective` para evitar posições negativas. Com isso, esta função retorna a imagem 1 transformada e os deslocamentos realizados em *x* e *y*.

No sétimo passo, a imagem 1 transformada é lida, juntamente com a imagem 2 original e os deslocamentos de translação em *x* e *y* feitos no passo anterior. A região onde a imagem 2 será inserida dentro da imagem 1 transformada é obtida, considerando os deslocamentos feitos no item anterior. Os *pixels* diferentes de zero na imagem 2 substituem estas posições na região de inserção obtida. Com isso, é retornada a imagem panorâmica completa, unindo ambas as imagens com alinhamento no espaço.

Por fim, no oitavo passo, foi criada a função `draw_correspondencies`, que recebe a imagem 1, a imagem 2, os pontos de interesse de 1 e de 2, a lista de pares válidos e a máscara binária de correspondência do *RANSAC*. Foi utilizada a função `cv2.drawMatches` do *OpenCV*, que recebe além dos parâmetros citados, configurações de cor das linhas e dos pontos, a máscara do e um parâmetro para desenhar apenas as correspondências, deixando a imagem mais limpa (`flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS`), retornando a imagem já desenhada.

Com isso, após a aplicação deste *pipeline*, a função `registry` retorna a imagem do panorama e a imagem das correspondências desenhadas, salvas no diretório de saída.

3 Resultados e Discussões

3.1 Transformações geométricas

Foram testadas a aplicação das transformações de escala e rotação com todas as imagens de entrada e com variação dos parâmetros, considerando alguns casos especiais. A Figura 4 mostra o resultado da aplicação da translação, mantendo a dimensão da saída igual a dimensão da entrada. A Figura 5 mostra o resultado da aplicação da rotação. Em ambos os casos, foi utilizada a interpolação por polinômios de *Lagrange*.

Como é possível perceber, a função foi implementada corretamente, realizando as transformações como o esperado. Ao utilizar -d como *max max*, a imagem escalada terá a dimensão resultante, como mostra a Figura 6.

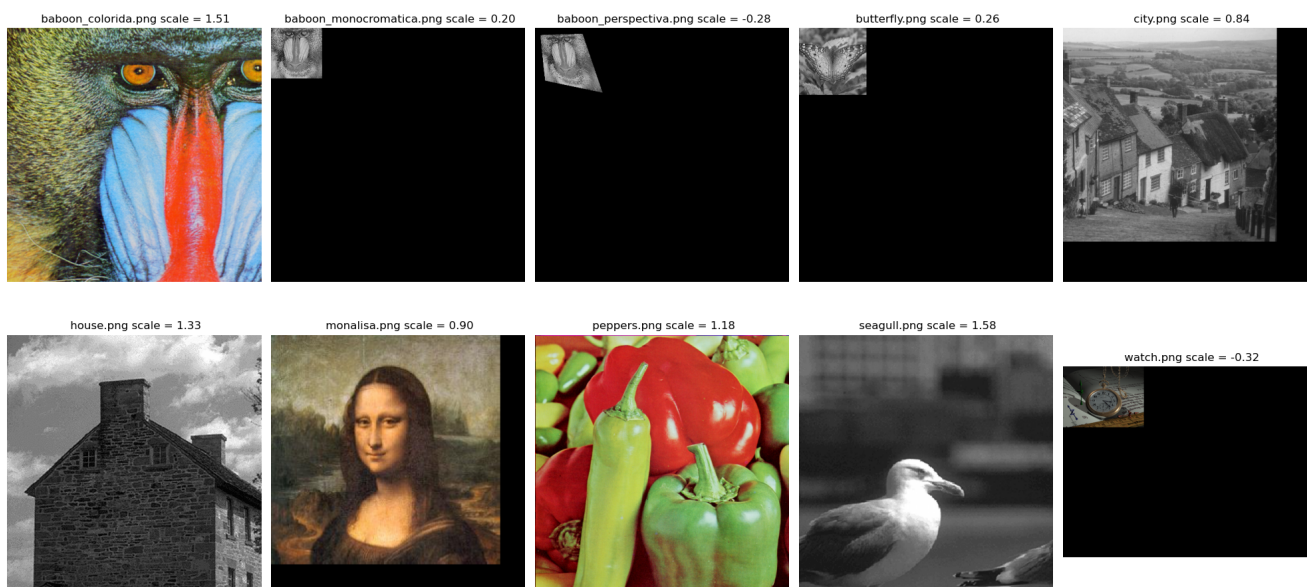


Figura 4: Transformações de escala aplicadas nas diferentes imagens. Valores negativos passados como fator de escala são ignorados, sendo considerado o seu módulo.

Após o teste com relação ao resultado das transformações, foi realizada uma comparação entre as diferentes funções de interpolação. Para visualizar a perda de detalhes, foi escolhida a transformação de escala com ampliação, como mostra a Figura 7. O mesmo teste foi realizado para a rotação (Figura 8).

Ao comparar as interpolações, é possível perceber que a interpolação por vizinho mais próximo teve uma execução mais rápida (0.10 segundos para a transformação total) devido à sua simplicidade, porém ocorreram algumas distorções e perdas de detalhes, tanto na rotação quanto na escala, principalmente criando um padrão serrilhado nas bordas retas.

Já a transformação bilinear, elimina esse caráter serrilhado, porém causou um certo borramento na imagem com algumas perdas de detalhes, com um tempo de processamento (0.14 segundos no total) maior com relação à interpolação por vizinhos. Já a transformação bicúbica, apesar do maior tempo de processamento dentre todos os métodos (0.39 segundos no total), eliminou o borramento e serrilhamento, preservando os detalhes finos das imagens. Por fim, a transformação por polinômios de *Lagrange*, apresentou resultados similares à bicúbica, porém com menor tempo de processamento (0.24 segundos no total).

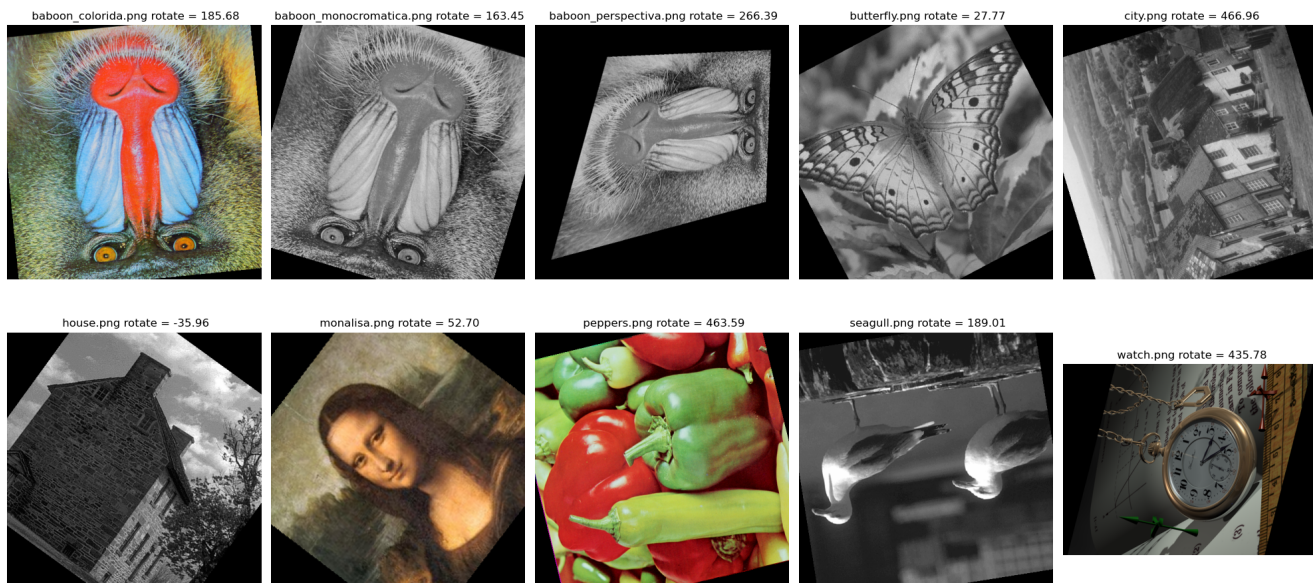


Figura 5: Transformações de rotações aplicadas nas diferentes imagens. Valores do ângulo (em graus) negativos passados representam rotações no sentido horário. Valores maiores que 360° representam mais de uma volta circular.



Figura 6: À esquerda, temos a imagem original. No centro, temos a imagem reescalada por um fator de 0.5, mantendo a dimensão da imagem original. À direita, mesma imagem, porém com a saída equivalente à dimensão reescalada, utilizando -d como max max.

No geral as transformações apresentaram resultados não tão discrepantes, com pequenas diferenças em detalhes finos.

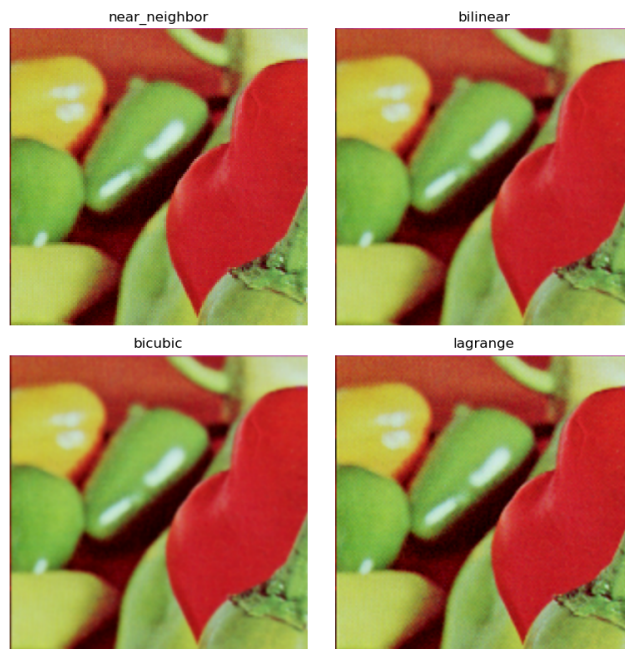


Figura 7: Comparação dos diferentes métodos de interpolação para a transformação de escala.



Figura 8: Comparação dos diferentes métodos de interpolação para a transformação de rotação.

3.2 Registro de Imagens

Após o desenvolvimento do programa, foram testadas para todas as imagens disponibilizadas a aplicação do algoritmo de registro. Foram comparados os resultados com as 3 técnicas de detecção e descrição implementadas, além de algumas variações no limiar de correspondências válidas. Com a variação do limiar, percebeu-se que para alguns

casos, como no *ORB*, menos correspondências são encontradas, portanto, valores muito altos de limiar não produzem resultados válidos. Em muitos casos, também, valores altos filtram muito as correspondências, causando erros por poucas correspondências. Valores mais baixos de limiar deixam as correspondências mais permissivas, podendo ocorrer correspondências errôneas.

As Figuras 9 e 10, mostram diferenças observadas ao variar o limiar. Limiares muito altos ou muito baixos geram imprecisões no registro.

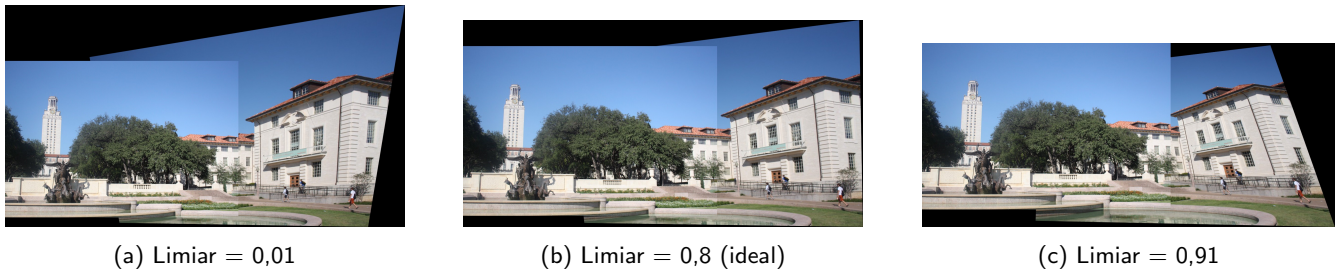


Figura 9: Imagens panorâmicas resultantes para diferentes limiares no método *ORB*.

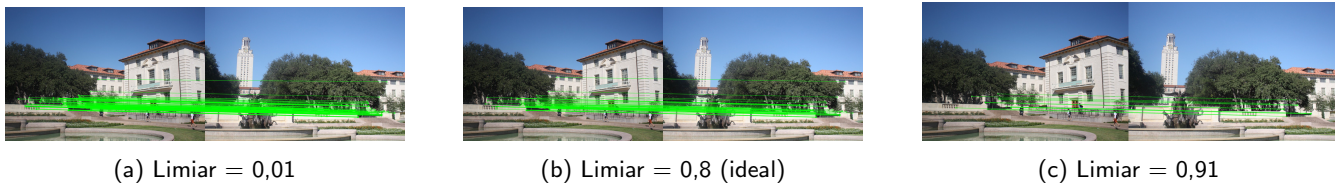


Figura 10: Imagens correspondências resultantes para diferentes limiares no método *ORB*. Maiores limiares produzem menos correspondências.

A comparação entre as técnicas *SIFT*, *ORB* e *BRIEF*, gerou as seguintes imagens. As matrizes de homografia são mostradas nas tabelas a seguir. Como é possível perceber a seguir, as técnicas tem algumas particularidades. Foi observado uma maior densidade de correspondências no *SIFT* e uma densidade mais baixa no *ORB*. No geral, o *ORB* necessitou de limiares mais baixos para obtenção de um bom resultado. A imagem 4 apresentou um maior desafio para o *BRIEF*, que produziu poucas correspondências, mesmo com limiares baixos. Por outro lado, a imagem 5 apresentou a maior dificuldade para todas os métodos, com poucas correspondências, produzindo resultados próximos a uma imagem. Nesta imagem, o melhor desempenho foi obtido pelo *BRIEF*, produzindo uma imagem panorâmica com preservação dos detalhes.

Com relação aos tempos de processamento do *pipeline* de registro, foi verificado que em média o *BRIEF* foi o mais demorado, enquanto o *ORB* foi o mais veloz, porém apresentou resultados menos robustos. O *SIFT* demonstrou resultados satisfatórios, com um tempo médio entre os 3.

Tabela 2: Tempo de processamento médio de cada método

Método	Tempo de computação (segundos)
SIFT	0.1975
ORB	0.07593
BRIEF	0.3807

Tabela 3: Matrizes de homografia obtidas para a Imagem 1.

Descritor	Matriz de Homografia		
SIFT	0.7651	0.0378	446.5823
	-0.1358	0.9122	76.1105
	-2.1194×10^{-4}	-3.1542×10^{-5}	1
ORB	0.8087	0.1069	428.0695
	-0.1332	0.9914	56.6225
	-2.0806×10^{-4}	4.3832×10^{-5}	1
BRIEF	0.7771	0.0406	446.1633
	-0.1331	0.9205	74.4981
	-2.0333×10^{-4}	-2.4300×10^{-5}	1

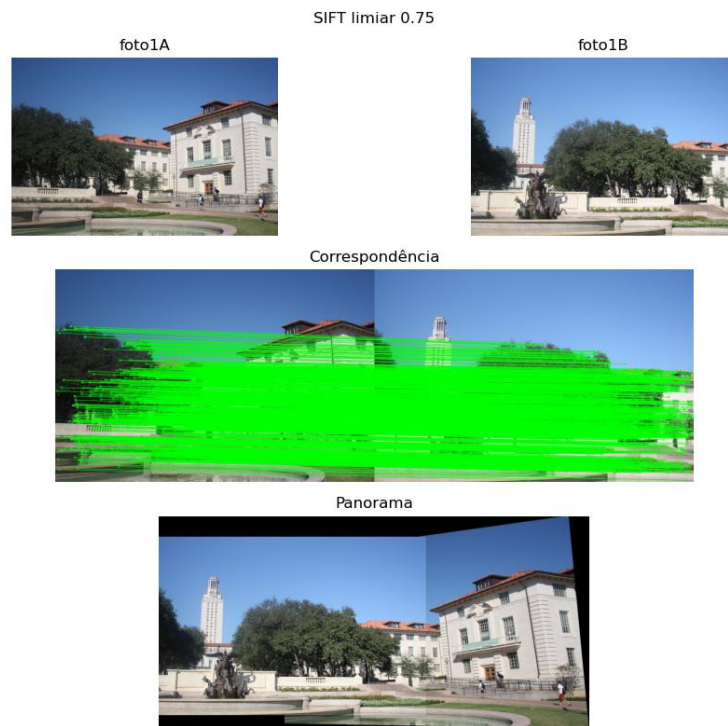


Figura 11: Imagem 1 com *SIFT* para limiar 0.75.

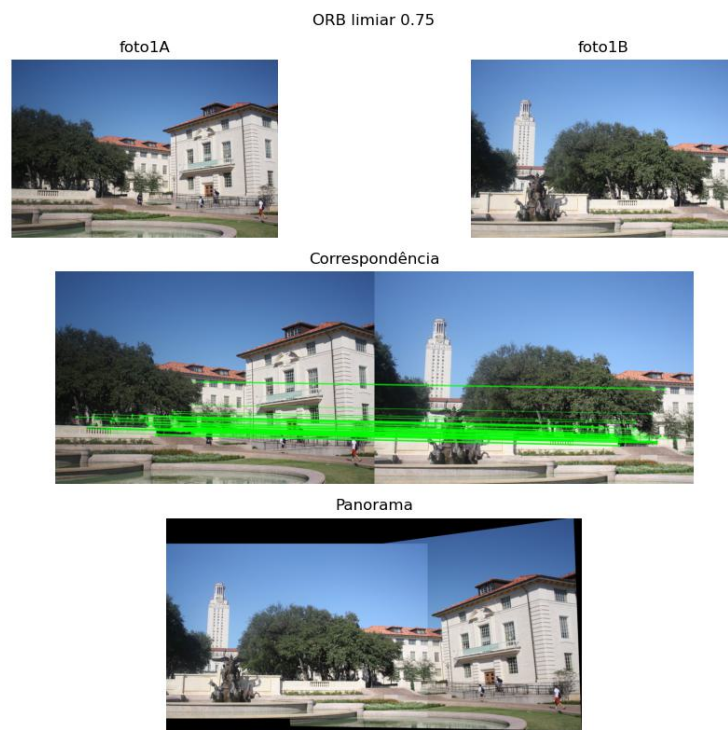


Figura 12: Imagem 1 com *ORB* para limiar 0.75.

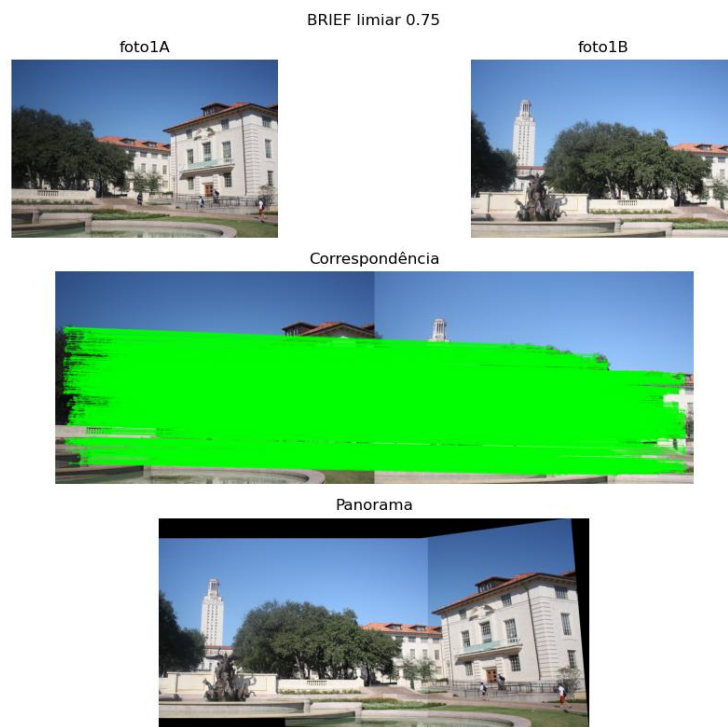


Figura 13: Imagem 1 com *BRIEF* para limiar 0.75.

Tabela 4: Matrizes de homografia obtidas para a Imagem 2.

Descritor	Matriz de Homografia			
SIFT	0.9998	-3.70×10^{-5}	463.0076	
	-1.32×10^{-5}	0.9999	0.0055	
	-2.10×10^{-7}	-1.84×10^{-8}	1	
ORB	0.9982	0.0018	463.0253	
	-2.71×10^{-4}	1.0006	-0.0402	
	-1.59×10^{-6}	2.85×10^{-6}	1	
BRIEF	1.0000	-6.82×10^{-6}	463.0031	
	-8.99×10^{-6}	0.9999	0.0249	
	5.17×10^{-8}	-3.55×10^{-8}	1	

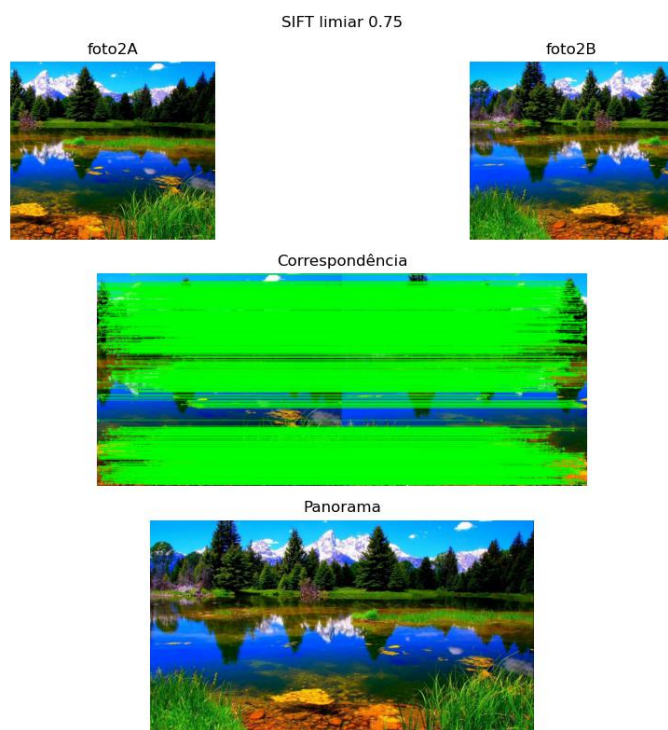


Figura 14: Imagem 2 com *SIFT* para limiar 0.75.

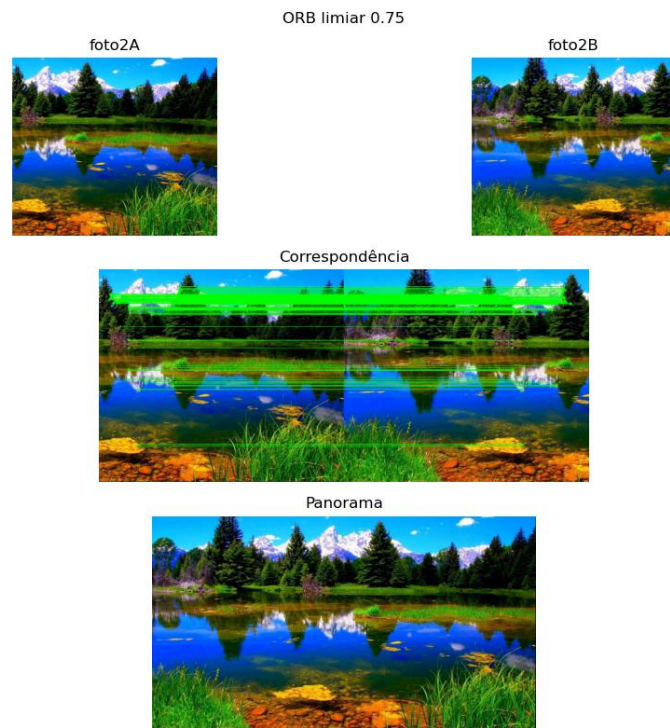


Figura 15: Imagem 2 com *ORB* para limiar 0.75.

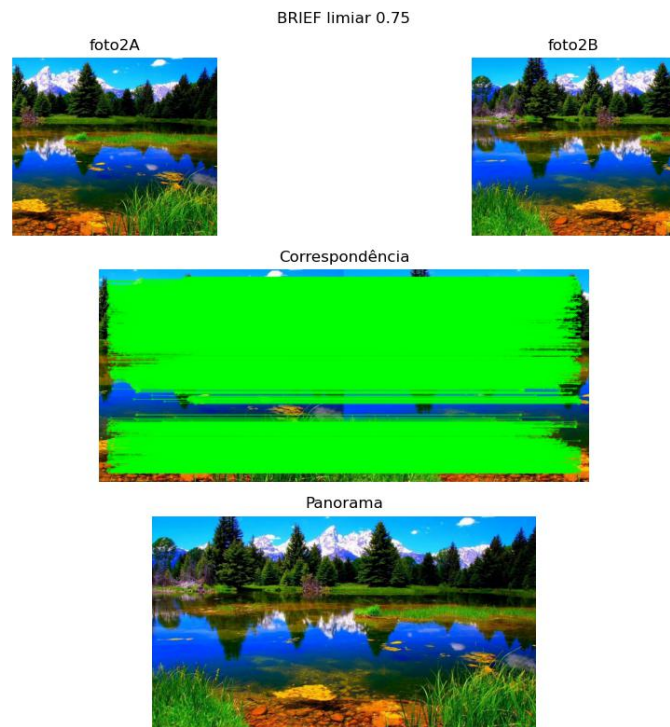


Figura 16: Imagem 2 com *BRIEF* para limiar 0.75.

Tabela 5: Matrizes de homografia obtidas para a Imagem 3.

Descritor	Matriz de Homografia		
SIFT	0.9884	-0.0198	32.3900
	0.0176	0.9822	-129.6508
	1.89×10^{-6}	-4.24×10^{-5}	1
ORB	0.9896	-0.0207	32.5183
	0.0179	0.9819	-129.6575
	3.06×10^{-6}	-4.24×10^{-5}	1
BRIEF	0.9892	-0.0033	29.9157
	0.0187	0.9935	-131.9051
	-1.21×10^{-5}	-8.47×10^{-7}	1

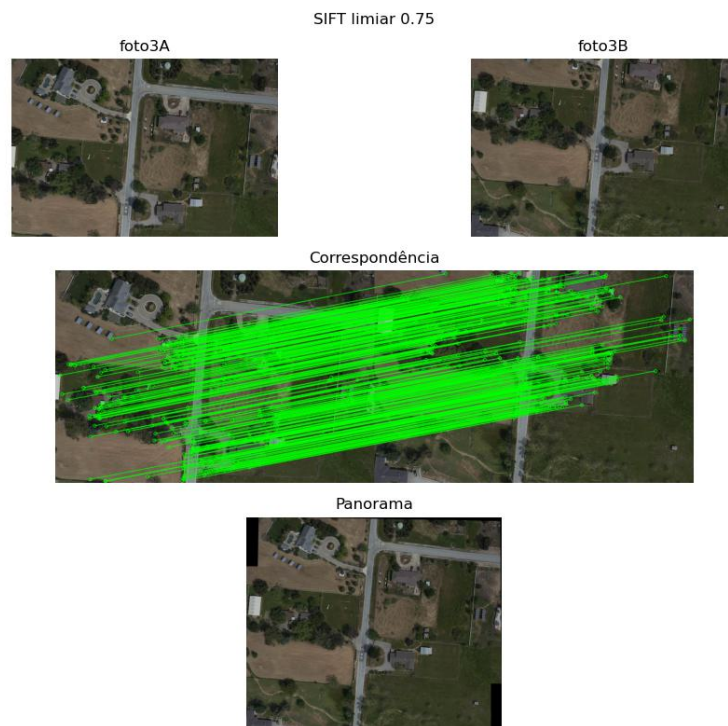


Figura 17: Imagem 3 com *SIFT* para limiar 0.75.

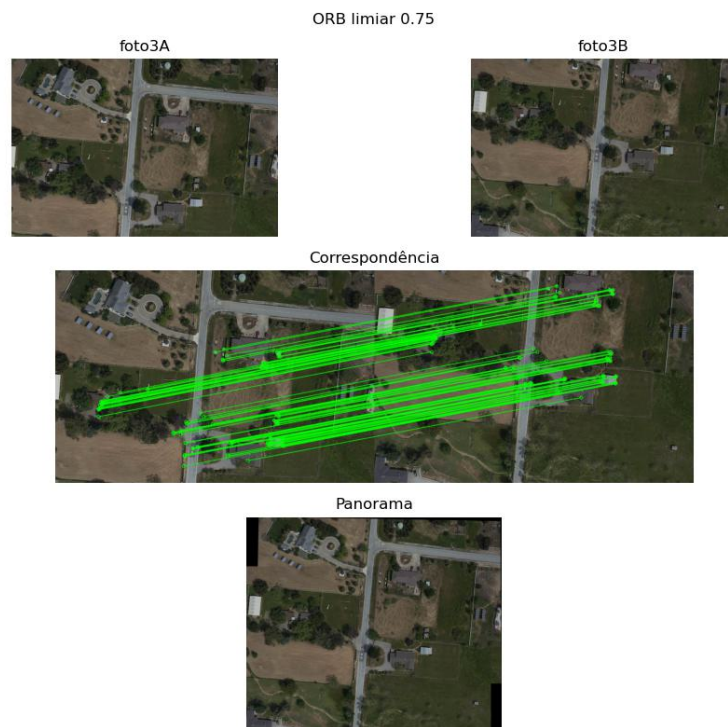


Figura 18: Imagem 3 com *ORB* para limiar 0.75.

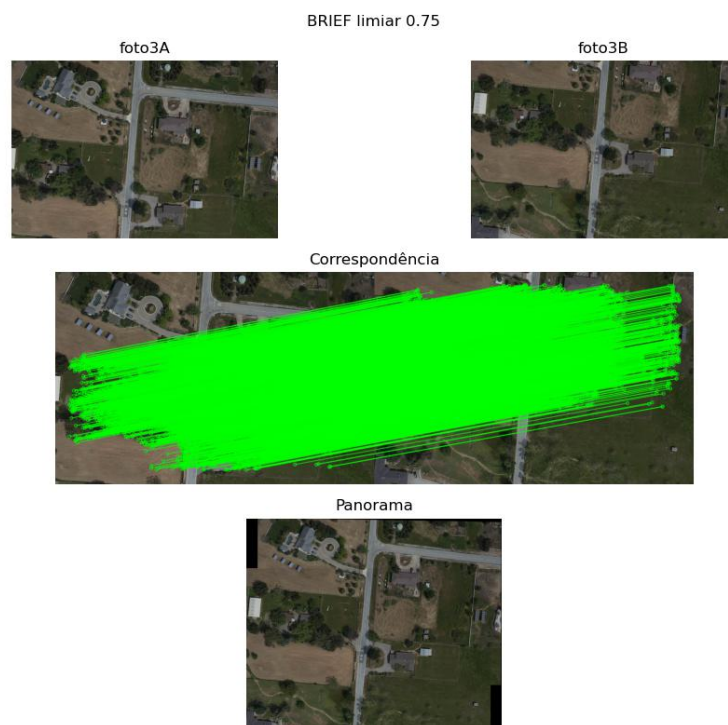


Figura 19: Imagem 3 com *BRIEF* para limiar 0.75.

Tabela 6: Matrizes de homografia obtidas para a Imagem 4.

Descritor	Matriz de Homografia			
SIFT	1.1554	1.0951	-219.7100	
	-1.0951	1.1558	213.0763	
	-1.95×10^{-6}	2.98×10^{-6}	1	
ORB	1.0634	0.9962	-188.4738	
	-1.0560	1.0852	209.1290	
	-8.72×10^{-5}	-1.06×10^{-4}	1	
BRIEF	-0.3780	-0.6364	120.3736	
	-0.7615	-1.2831	241.1528	
	-3.16×10^{-3}	-5.32×10^{-3}	1	

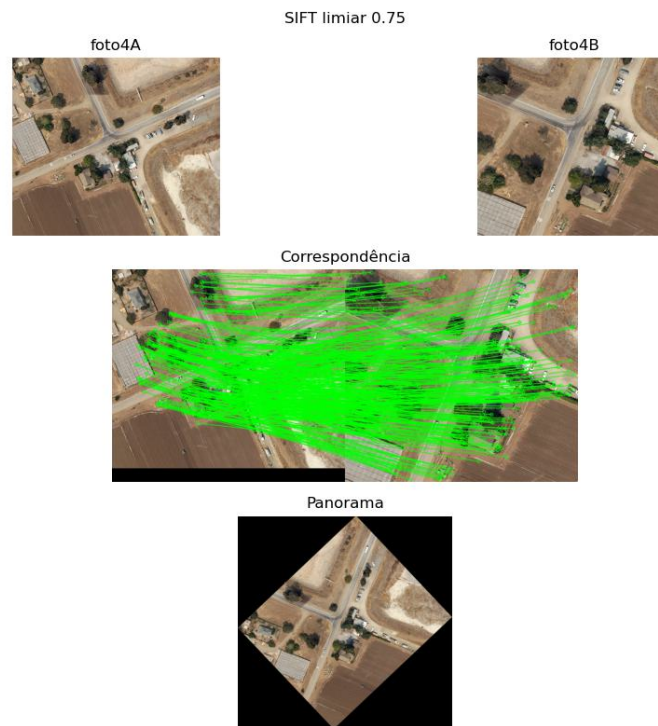


Figura 20: Imagem 4 com *SIFT* para limiar 0.75.



Figura 21: Imagem 4 com *ORB* para limiar 0.75.



Figura 22: Imagem 4 com *BRIEF* para limiar 0.75.

Tabela 7: Matrizes de homografia obtidas para a Imagem 5.

Descritor	Matriz de Homografia		
SIFT	-0.8213	0.0018	509.4972
	-0.1147	-0.0051	73.6084
	-1.61×10^{-3}	7.39×10^{-6}	1
ORB	0.9702	0.0077	38.4441
	0.0139	1.1093	90.9795
	-2.21×10^{-4}	5.05×10^{-4}	1
BRIEF	0.9818	-0.0548	40.5125
	0.0538	1.0103	91.4300
	-2.83×10^{-5}	5.47×10^{-5}	1

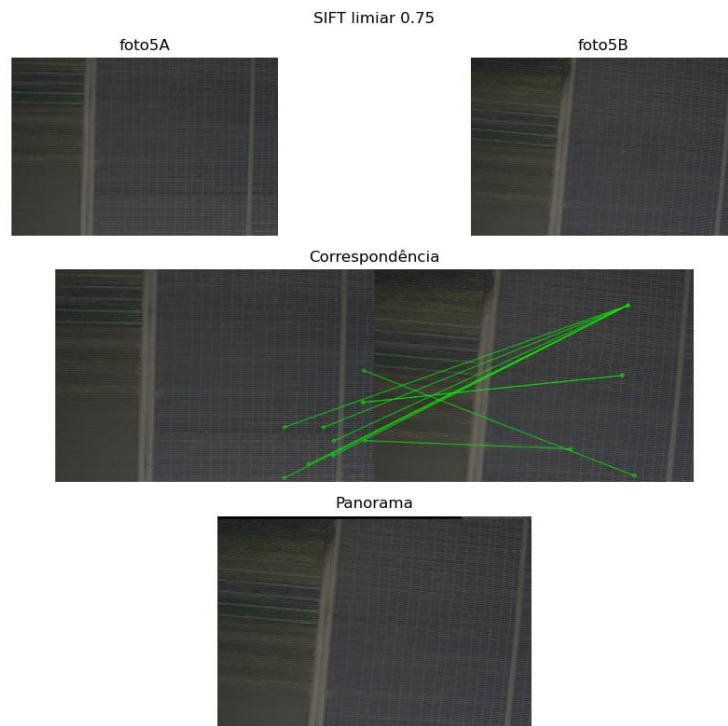


Figura 23: Imagem 5 com *SIFT* para limiar 0.75.

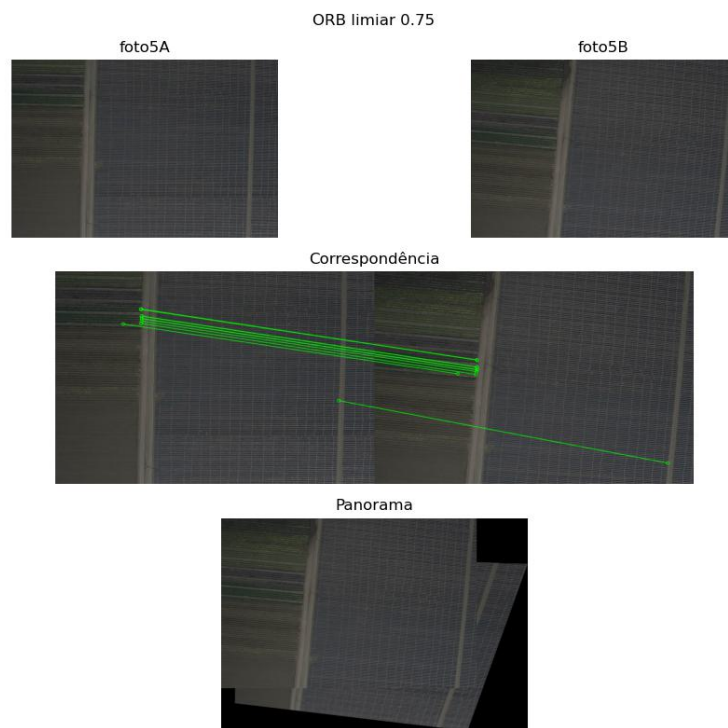


Figura 24: Imagem 5 com *ORB* para limiar 0.75.

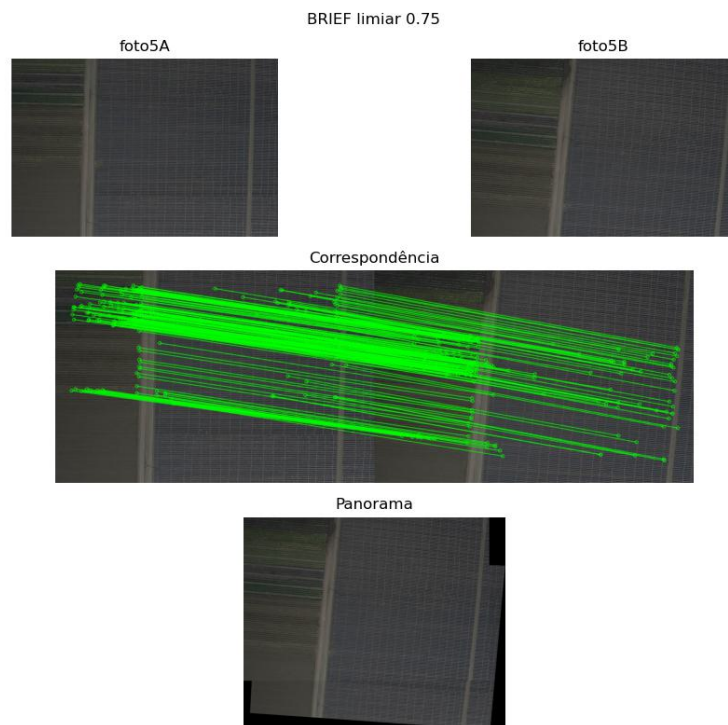


Figura 25: Imagem 5 com *BRIEF* para limiar 0.75.

4 Conclusões

Neste trabalho, foi possível implementar sem o uso de bibliotecas, transformações de escala e rotação, observando os desafios que surgem com essas técnicas. O principal problema que surge está ligado aos resultados contínuos para os novos posicionamentos dos *pixels*, que causam problemas para associar aos valores das imagens originais. Por isso, foram implementadas diferentes técnicas de interpolação, responsáveis em inferir o valor do *pixel* após a transformação com base em aproximações por meio de vizinhança. Com isso, foi observado algumas características dos métodos, incluindo qualidade na preservação de detalhes e tempo computacional.

Após estes experimentos, foi implementado um *pipeline* de registro, de forma a obter, a partir de duas imagens de uma mesma localização, pontos em comum, que representam mesmas estruturas nas imagens. Os pontos considerados correspondentes, são base para transformações de perspectiva, que distorcem uma imagem para o alinhamento destes pontos, resultando em uma imagem panorâmica que combina ambas as imagens. Foi comparado o desempenho dos descritores *SIFT*, *ORB* e *BRIEF* nesta tarefa, comparando os resultados obtidos e o tempo computacional.

Referências

- [1] Anaconda, Inc. Miniconda release notes (version 25.9.1-1). <https://www.anaconda.com/docs/getting-started/miniconda/release-notes#miniconda-25-9-1-1>, 2025. Acesso em: 04 abr. 2026.
- [2] NumPy Developers. Numpy documentation. <https://numpy.org/doc/2.4/reference/index.html>, 2025. Acesso em: 04 abr. 2026.
- [3] Hunter, et al. Opencv documentation. <https://matplotlib.org/stable/index.html>, 2025. Acesso em: 04 abr. 2026.
- [4] OpenCV. Opencv documentation. <https://docs.opencv.org/4.12.0/>, 2025. Acesso em: 04 abr. 2026.
- [5] Hélio Pedrini. Banco de imagens png. https://www.ic.unicamp.br/~helio/imagens_png/, 2025. Acesso em: 20 jun. 2026.
- [6] Hélio Pedrini. Banco de imagens para registro. https://www.ic.unicamp.br/~helio/imagens_registro/, 2025. Acesso em: 20 jun. 2026.
- [7] Hélio Pedrini. Trabalho1. <http://www.ic.unicamp.br/~helio/disciplinas/M0443/trabalho4.pdf>, 2026. Acesso em: 21 jun. 2026.
- [8] Numpy. Função meshgrid. <https://numpy.org/doc/stable/reference/generated/numpy.meshgrid.html>, 2025. Acesso em: 20 jun. 2026.
- [9] Hélio Pedrini. Aula registro. http://www.ic.unicamp.br/~helio/disciplinas/M0443/aula_registro.pdf, 2026. Acesso em: 21 jun. 2026.
- [10] David G Lowe. Object recognition from local scale-invariant features. In *Proceedings of the seventh IEEE international conference on computer vision*, volume 2, pages 1150–1157. Ieee, 1999.
- [11] Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua. Brief: Binary robust independent elementary features. In *European conference on computer vision*, pages 778–792. Springer, 2010.
- [12] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. Orb: An efficient alternative to sift or surf. In *2011 International conference on computer vision*, pages 2564–2571. Ieee, 2011.
- [13] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [14] OpenCV. Feature matching + homography to find objects. https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_feature2d/py_feature_homography/py_feature_homography.html, 2016. Acesso em: 21 jun. 2026.
- [15] OpenCV. Geometric transformations of images. https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_geometric_transformations/py_geometric_transformations.html, 2016. Acesso em: 21 jun. 2026.
- [16] Edward Rosten and Tom Drummond. Machine learning for high-speed corner detection. In *European conference on computer vision*, pages 430–443. Springer, 2006.
- [17] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: Speeded up robust features. In *European conference on computer vision*, pages 404–417. Springer, 2006.